

Verilator



SHAKTHI KANNAN

Verilator is a Verilog hardware description language (HDL) simulator that can compile synthesisable Verilog code into C++ or SystemC. It is designed primarily for high-performance simulations, and supports simple assertions and code-coverage analysis. It is released under GNU LGPL/Perl artistic licence. You can install it on Fedora 20 (x86_64) using the following:

```
$ sudo yum install verilator
```

Consider a simple hello.v example.

```
module hello;

    initial begin
        $display("Hello!");
        $finish;
    end

endmodule
```

A C++ wrapper file is written to test drive the hello module.

```
#include "Vhello.h"
#include <verilated.h>

int
main (int argc, char **argv, char **env)
{
    Verilated::commandArgs(argc, argv);

    Vhello* top = new Vhello;

    while (!Verilated::gotFinish()) { top->eval(); }

    exit (0);
}
```

You can compile the above code with Verilator and generate required simulation files with `--cc`.

```
$ verilator --cc hello.v --exe main.cpp
```

Before running `main.cpp`, you need to install GCC and GCC-C++ in your system.

This produces `obj_dir` with Makefiles and C++ code. These generated files can then be compiled using the following:

```
$ cd obj_dir
$ make -j -f Vhello.mk Vhello

g++ -I. -NMMD -I/usr/share/verilator/include \
    -I/usr/share/verilator/include/vltstd -DVL_PRINTF=printf \
    -DVM_TRACE=0 -DVM_COVERAGE=0 \
```

```
-c -o main.o ../main.cpp
g++ -I. -NMMD -I/usr/share/verilator/include \
    -I/usr/share/verilator/include/vltstd -DVL_PRINTF=printf \
    -DVM_TRACE=0 -DVM_COVERAGE=0 \
    -c -o verilated.o /usr/share/verilator/include/
    verilated.cpp
/usr/bin/perl /usr/share/verilator/bin/verilator_includer \
    Vhello.cpp > Vhello_ALLCIs.cpp
/usr/bin/perl /usr/share/verilator/bin/verilator_includer \
    Vhello_Syms.cpp > Vhello_ALLSup.cpp
g++ -I. -NMMD -I/usr/share/verilator/include \
    -I/usr/share/verilator/include/vltstd -DVL_PRINTF=printf \
    -DVM_TRACE=0 -DVM_COVERAGE=0 \
    -c -o Vhello_ALLSup.o Vhello_ALLSup.cpp
g++ -I. -NMMD -I/usr/share/verilator/include \
    -I/usr/share/verilator/include/vltstd -DVL_PRINTF=printf \
    -DVM_TRACE=0 -DVM_COVERAGE=0 \
    -c -o Vhello_ALLCIs.o Vhello_ALLCIs.cpp
    Archiving Vhello_ALL.a ...
ar r Vhello_ALL.a Vhello_ALLCIs.o Vhello_ALLSup.o
ar: creating Vhello_ALL.a
ranlib Vhello_ALL.a
g++ main.o verilated.o Vhello_ALL.a -o Vhello -lm
-lstdc++ 2>&1 | c++filt
```

You can test the hello module with the following command:

```
$ cd ..
$ ./obj_dir/Vhello

Hello!
- hello.v:5: Verilog $finish
```

Verilator accepts a number of arguments as options. `-V` lists the version of the software and provides a summary of configuration and environment settings. A pre-processing output of the code is produced with `-E`, without actually compiling or generating any code. This is illustrated below.

```
$ verilator -cc hello.v -E
```

```
`line 1 "hello.v" 1
module hello;

`line 3 "hello.v" 0
    initial begin
        $display("Hello!");
        $finish;
    end

endmodule

`line 9 "hello.v" 2
```

`-CFLAGS` allows the user to override any C++ compiler flags during the build process. For example,

```
$ verilator -cc hello.v -CFLAGS -O0 --exe main.cpp
```

The respective flags are passed to the compiler in the